

P1130R2

Module Resource Dependency Propagation

Published Proposal, 2026-03-27

This version:

https://thephd.dev/_vendor/future_cxx/papers/d1130.html

Author:

[JeanHeyd Meneide](#)

Audience:

EWG, SG15

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Latest:

https://thephd.dev/_vendor/future_cxx/papers/d1130.html

Abstract

This proposal provides a way of tracking dependencies at preprocessor time and allows modules to further understand those dependencies by understanding which are available to consumers of those modules ("public", marked by the keyword `export`) and which are module or translation unit-internal ("private", not marked by any keyword).

Table of Contents

1 Revision History

- 1.1 Revision 2 - March 28th, 2026
- 1.2 Revision 1 - January 21st, 2019
- 1.3 Revision 0 - November 26th, 2018

2 Motivation

3 Design

3.1 Global Module Fragment?

3.2 Example Cases

3.2.1 Semi-Related Case: Reachability of Data with `#embed`

3.2.2 Modules and Exported Functions

3.2.2.1 Case 0: `#depend` in the Module but not in the Consumer

3.2.2.2 Case 1: `#depend` in the Consumer but not in the Module

3.2.2.3 Case 2: `#depend` in the Module but not in the Consumer

3.2.3 What About Templates?

4 Proposed Changes to the Standard

4.1 Intent

4.2 Proposed Feature Test Macro

4.3 Proposed Wording

5 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 2 - March 28th, 2026

- Rework and put forward in response to `std::embed` only working within a single translation unit.
- Much simpler syntax and design.
- There is no wording until the initial design discussion happens in EWG / SG15 for the direction here.

§ 1.2. Revision 1 - January 21st, 2019

- Add words of power, "resource location".
- Clarify wording around it not being a hard error.

- Track changes to [\[p1103\]](#) accordingly.
- Fix description in motivation of the use of `require`.
- Add HRESOURCE example.

§ 1.3. Revision 0 - November 26th, 2018

- Initial release.

§ 2. Motivation

The only way to declare a dependency in C++ currently is to use a `#include` statement. With Modules coming to C++, dependency information was greatly enhanced by preambles and globule module fragments which allow for both the compiler and build system to understand physical and semantic layout of code. However, there is still a problem area that C++ has not addressed that people in the brave new Modules ecosystem want to answer: external dependency information for Modular C++.

Part of this issue was tackled as part of [p1040 `std::embed`](#) and its `#depend` statement, but after a vote during by the Evolution Working Group in the 2025 Sofia C++ Meeting, `#depend` was restricted to be translation unit-specific while a solution for dependencies was explored. This paper is the exploration requested by EWG and by proponents of SG15. Given this example, as of March 2026 EWG Consensus, the comments show which actions are well-formed or ill-formed:

`m0.cpp`:

```
export module m0;

#depend "header.bin"

import <embed>;
import combine;

export consteval auto f0(std::string_view f) {
    const auto h = std::embed("header.bin");
    const auto t = std::embed(f);
    return combine(h, t);
}
```

m1.c++m:

```
export module m1;

#depend "default.stuff"

import m0;

export consteval auto f1(std::string_view f = "default.stuff") {
    return f0(f);
}

export consteval auto getpath() {
    return "default.stuff";
}
```

translation_unit0.c++:

```
import m1;
import print;
import <embed>

#depend "coolstuff.bin"

int main() {
    print(f1("coolstuff.bin")); // [0] fails
    print(f1());                // [1] fails
    std::embed("header.bin");    // [2] fails
    std::embed(getpath());       // [3] fails
    std::embed("coolstuff.bin"); // [4] ok
}
```

All of [0], [1], [2], and [3] fail because each of them is trying to access dependencies that are part of translation units outside of the current translation unit. This was an intentional decision because of module privacy. To make it more clear why this decision was taken, consider a more appropriately named set of items that illustrate the issue:

deps.cppm:

```
export module deps;

#depend <sdk/private/**>
#depend <sdk/everything/**>
```

main.cpp:

```
import deps;

int main () {
    std::embed("sdk/everything/meow.wav"); // ok
    std::embed("sdk/private/super_secret_sauce.mix"); // uh-oh, ok?!
    return 0;
}
```

Using a strategy that said "all `#depend` statements accumulate in the module and are passed down" means that the "private" part of the SDK that is only available (or meant to be available) to the module is now exposed. The user should not be able to access the `sdk/private` portion, but the `sdk/everything` directory should be available to downstream consumers.

Thusly, we propose adding the `export` identifier as a (contextual?) keyword to solve this problem, and make clear which depended-on things are meant to be propagated to public consumers, and give build tools the chance to properly accumulate a list of file and folder matchers that let them know to rebuild downstream dependents appropriately.

§ 3. Design

The solution we are proposing is simple, here. Any `#depend` not marked as `export` is not combined into the module.

deps.cppm:

```
export module deps;

#depend <sdk/private/**> // module and translation unit-internal
#depend export <sdk/everything/**> // module public
```

main.cpp:

```
import deps;

int main () {
    std::embed("sdk/everything/meow.wav"); // previously ill-formed -- ok, no
    std::embed("sdk/private/super_secret_sauce.mix"); // ill-formed
    return 0;
}
```

The syntax piggybacks off of [p1040 std::embed](#), so we hope that this proposal succeeds alongside or after [p1040 std::embed](#).

An open question is if we want to have **two** keywords for **three** levels: module-internal, translation unit-internal, and then module-public. It does not seem very beneficial to have this kind of differentiation. It's also unclear what "module-internal" would mean -- there is still no standards-mandated way to identify all of the translation units that make up a module, definitively, by the end of preprocessor time without additional external assistance. This is different from having just translation unit-internal and module-public, as module dependencies can at the very least be traced between each other based on `import` statements and ordering can be established properly before the `import`-ing modules are done.

§ 3.1. Global Module Fragment?

We are not proposing this information should go in the global module fragment. Such a requirement is unnecessary and does not help when replacing cases of `#include ...` or `#embed ...` which already can appear anywhere in a translation unit. There is no reason for this to be added explicitly and only to the global module fragment, so this proposal does not require us to put it there.

The ownership of dependencies is squarely for the module. The "inheriting" of dependencies only comes from properly `export`-marked `#depend` directives. All of this information is available after preprocessing, which is exactly the phase we want input dependencies in-general (such as `#include` and `#embed`) to be resolved. `#depend` should be no different in this regard and should not impose a burden greater than either `#include` or `#embed` already do in the current modules ecosystem.

§ 3.2. Example Cases

We evaluate the following code sample with minor tweaks to illustrate the reachability and dependency behavior for Modules with this paper as compared to the status quo.

§ 3.2.1. Semi-Related Case: Reachability of Data with `#embed`

First, a question was brought up about the "reachability" of certain kinds of data from preprocessor directives / constant expressions. A simple example is this:

`mod.c++.m.`

```
export module mod;

constexpr export const unsigned char x[] = {
    #embed <data.bin>
};
```

The question came up is it possible to "stream" the data from `<data.bin>` and otherwise include that information in the module when it is imported to downstream consumers, such that an implementation could not embed the initializer in the module's code but instead embed just a reference to the file. The answer is yes. It is no different from the following case:

`mod-oldstyle.c++.m.`

```
export module mod;

export constexpr const unsigned char x[] = {
    #include <data.bin.h>
};
```

Where a pre-build tool generated the header data from `"data.bin"` and required a user to interact with the generated header. Whether an implementation decided to make available the sequence of integer literals generated by the pre-build tool or try to be clever with a streaming solution was always a matter of QoI. `#embed` offers both the user and the implementation a chance to represent what they are trying to do better than the status quo, and gives that information by the end of preprocessing.

§ 3.2.2. Modules and Exported Functions

Here is an example of a translation unit that is consuming a module `m`. We will use various flavors of this with `#depend` statements and `std::embed` calls to illustrate what is allowed and what is not allowed.

`m.c++m:`

```
export module m;

#include <embed>

void xyz() {
    std::embed("foo.bin"); // (1)
}

export void abc() {
    xyz();
}
```

`main.c++:`

```
import m;

#include <embed>

int main() {
    std::embed("foo.bin"); // (2)
    abc();
}
```

In this plain example, both (1) and (2) are ill-formed for the obvious reason that there is no `#depend` directive greenlighting access. This allows us to set up cases based on the use of the `#depend` directive.

§ 3.2.2.1. Case 0: `#depend` in the Module but not in the Consumer

`m.c++m:`

```
export module m;

#include <embed>

#depend "foo.bin"

void xyz() {
    std::embed("foo.bin"); // (1)
}

export void abc() {
    xyz();
}
```

`main.c++:`

```
import m;

#include <embed>

int main() {
    std::embed("foo.bin"); // (2)
    abc();
}
```

Under the design of this paper, (1) is legal but (2) is ill-formed. The reason is simple: `#depend`s do not exit the module / translation unit they are contained inside of. This allows relying on private dependencies that may only be available to the module's translation units during compilation without "poisoning" or "contaminating" downstream users.

§ 3.2.2.2. Case 1: `#depend` in the Consumer but not in the Module

`m.c++m:`

```
export module m;

#include <embed>

void xyz() {
    std::embed("foo.bin"); // (1)
}

export void abc() {
    xyz();
}
```

`main.c++:`

```
import m;

#include <embed>

#depend "foo.bin"

int main() {
    std::embed("foo.bin"); // (2)
    abc();
}
```

Under the design of this paper, (1) is ill-formed but (2) is legal. `#depend`s are still private and do not flow upwards. Even if -- somehow -- the `#depend` was placed before the `import m;`, it would not "leak" that information into importing modules. It also cannot do this even if the `export` keyword was added and `main.c++` looked like this:

`main.c++:`

```
import m;

#include <embed>
```

```
#depend export "foo.bin"

int main() {
    std::embed("foo.bin"); // (2)
    abc();
}
```

(1) is still ill-formed and (2) is still legal.

§ 3.2.2.3. Case 2: `#depend` in the Module but not in the Consumer

`m.cpp`:

```
export module m;

#include <embed>

#depend export "foo.bin"

void xyz() {
    std::embed("foo.bin"); // (1)
}

export void abc() {
    xyz();
}
```

`main.cpp`:

```
import m;

#include <embed>

int main() {
    std::embed("foo.bin"); // (2)
    abc();
}
```

Both (1) and (2) are well-formed here. The `export` on the `#depend` directive means that the module exports that specific dependency pattern to its modules consumers. This makes the `std::embed` calls in both locations legal and safe to perform. A modification of `"foo.bin"` would trigger a reconsideration of inputs in the module and any of its consumers.

§ 3.2.3. What About Templates?

The answers if both `abc` and `xyz` were templated with `template <typename T>` would not change the answers to these questions, even if the inputs to `std::embed` were to be dependent types/values that relied on the template parameters. The reason for this is very simple: dependency information flowing backwards into the templates of a module would result in templates that could easily violate ODR rules by behaving differently based on where it was instantiated in a downstream template. Therefore, to end up in a situation that is not like `__COUNTER__` or similar problems, that case should remain banned. That is:

`m.c++m:`

```
export module m;

#include <embed>

template <typename T>
void xyz() {
    std::embed("foo.bin"); // (1)
}

template <typename T>
export void abc() {
    xyz<T>();
}
```

`main.c++:`

```
import m;

#include <embed>

#depend "foo.bin"

int main() {
```

```
std::embed("foo.bin"); // (2)
abc<T>();
}
```

(1) is still ill-formed here while (2) is legal, just like in the original non-templated code. This is imperative to avoid needing to "inject" information from downstream input dependencies into upstream consumers. This is to preserve the integrity of modular boundaries and module ownership and to prevent `#depend` statements from having information from consumers "flow up" into modules and poison them in a way that could result in many different problems, such as ODR traps or template substitutions failures that only happen some of the time. Determinism is an important quality of the system.

§ 4. Proposed Changes to the Standard

No wording is being provided just yet as we need to wait for [p1040] to get closer to proper review. We do list the feature test macro and the intent, just so individuals reading this paper understand what we want. The intent is relative to the latest [p1040] draft and the latest C++ working draft.

§ 4.1. Intent

The intent of the upcoming wording is to:

- enable a `#depend` to accumulate dependencies for a translation unit if it is not immediately followed by the keyword `export`;
- enable a `#depend` to accumulate dependences **that are propagated** to other modules if it is immediately followed by the keyword `export`.

§ 4.2. Proposed Feature Test Macro

The proposed feature test macro for this is `__cpp_pp_depend_export`.

§ 4.3. Proposed Wording

NOTE: The wording will not exist until a decision is rendered by EWG.

§ 5. Acknowledgements

Thanks to Wyatt Childers, Michael Spencer, and Hana Dušíková for guiding this direction with their important feedback on [\[p1040\]](#).

§ References

§ Informative References

[P1040]

JeanHeyd Meneide. *[std::embed](#)*. October 12th, 2018. URL: <https://wg21.link/p1040>

[P1103]

Richard Smith. *[Merging Modules](#)*. November 26th, 2018. URL: <https://wg21.link/p1103>